

Build your own Agentic GTM



Three individual workshops. Your company, your accounts, your results.

Build your own Agentic GTM

Participant booklet. October 2026.

Use one local project for all three sessions. The starter code is prepared. Your work is to adapt it to your company, inspect the outputs and run only the actions you approve.

Begin with a useful file

1. Run this command in Codex, Claude Code or your terminal:

```
git clone https://github.com/f-o-x11/agentic-gtm-workshop.git
```

- 2. Open agentic-gtm-workshop as your local Codex or Claude Code project.
- 3. Open START-HERE.html in your browser and copy its complete prompt into the local agent.
- 4. Supply your website. Add your buyer segment, offer and two target domains if known.
- 5. Review company-motion/COMPANY.md before continuing.
- 6. Open prompts.html for the next exercise. The agent stops after each step.

Program commands run inside code/. Your company files live in company-motion/, beside it. Filled credentials and configuration stay in a separate private folder. The startup launcher saves a verified Python 3.10 or newer executable and reuses it. If none exists, use the official installer it names. No Homebrew is required.

Build the full local workflow with questions

For a build outside the guided class, paste this into the cloned local project:

Read BUILD-MY-GTM.md and follow its Build my workflow instructions. Ask me one question at a time, use my answers to adapt this project, and complete all local preparation that does not need another answer. Start with my company website. Keep external actions paused until I review their exact recipients, messages and costs.

This mode prepares the supported workflow. A channel becomes working only after its real provider result is checked.

What you can finish

Session	Guided work	Outcome
Part 1	90 minutes plus30 minutes individual help	Company brief, two reviewed account pages and a reusable skill. No paid API.
Part 2	90 minutes plus30 minutes individual help	Exact email previews,own-sender Sent test,and bounded ten-company pilot when prerequisites pass.
Part 3	90 minutes plus30 minutes individual help	Actual outcomes, next bounded cycle, pause/recovery and a saved operating record. Extra channels optional.

If an account or source is blocked, keep useful drafts and record the missing connection. Prepared, queued, sent, replied and booked are different results. A failed connection does not become successful because a token file exists.

Prepare only the tools you need

Part 1 needs your local coding agent, file/web access, website and supported Python. Part 2 needs owned Google OAuth, fresh complete exclusions, actual employer evidence and fresh valid work-email evidence. A finder is optional. OAuth may need your Workspace administrator. Start that setup early. See PREWORK.md and SETUP-SEQUENCE.md.

Part 3 uses the same core project. Gojiberry, Loop & Tie, Netlify, Calendly, CRM and Metadata are optional according to the extension you use. API-Setup-Guide.pdf and API-CONNECTIONS.md explain exact supported operations, credits, account ownership and proof.

Part 1: Build two account pages and save the skill.



Finish with

- Your company facts and paused operating instructions.
- Two sourced target briefs and two reviewed pages.
- One saved page skill successfully reused.
- A progress record and the exact next setup task.

Before this session

- Install or open local Codex or Claude Code with file and web access.
- Run `git clone https://github.com/f-o-x11/agentic-gtm-workshop.git`. Open `agentic-gtm-workshop` as the project; run later commands from `code/`.
- Use your website, buyer segment, offer and two real target domains. If unsure, the agent asks only for missing information.
- Python is checked by the launcher. If no supported Python exists, use its named official installer.
- No Google Cloud project, research API or paid channel is needed for Part 1.

Five chapters

- 1. See Metadata's working examples: See the actual page, email and gift examples.
- 2. Set up your company: `COMPANY.md`, saved setup and local rules.
- 3. Choose two target companies: Two target briefs and `TARGETS.md`.
- 4. Build and check your first page: First HTML page and `PAGE-REVIEW.md`.
- 5. Save the skill. Build page two.: `SKILL.md`, second page and completed file check.

Chapter 1: See Metadata's working examples

See the actual page, email and gift examples.

Build your own Agentic GTM

Reference: p1-cover. Main workshop.

Part 1. Build two pages for your target companies.

This is how I built Metadata's GTM motion

Reference: p1-results. Main workshop.

Run git clone <https://github.com/f-o-x11/agentive-gtm-workshop.git>. Open the folder in local Codex or Claude Code. Keep the booklet and prompts.html open.

The workflow I run at Metadata

Reference: p1-v5-tree. Main workshop.

Your agent researches buyers and prepares work. The local program checks the recipient, performs an approved action and records the provider result.

I'm Gil. I run Metadata.

Reference: p1-about. Main workshop.

Gil Allouche. CEO, Metadata.io. I built the workflow our marketing team uses. I use Codex to run it. Today, build two pages for your target companies.

A real page, email and gift

Reference: p1-real-outputs. Main workshop.

Metadata built a page for Zuora, sent an email to Forum One and sent a gift invitation to Datarails. These are different examples.

Build these files for your own company

Reference: p1-finish. Main workshop.

Use your company and two real targets. Open Participant-Booklet.pdf for instructions and prompts.html to copy each full prompt. No API keys are needed for Part 1.

Chapter 2: Set up your company

COMPANY.md, saved setup and local rules.

Open the cloned folder. Build your company brief.

Reference: p1-install. Main workshop.

Use the project you cloned on slide 2. Open START-HERE.html, add your website and copy the full startup prompt.

Time: 12 minutes. Difficulty: Easy.

Tools: Local Codex or Claude Code; file and web access. No API credits..

Do this:

1. Open agentive-gtm-workshop in local Codex or Claude Code.
2. Open START-HERE.html in your browser.
3. Add your website. Copy the full prompt into your local agent.
4. Read COMPANY.md. Keep using this project.

Copy the complete prompt from prompts.html#p1-install. Paste it into the same local Codex or Claude Code project.

Help me build my company's GTM workflow one step at a time.
Website: [YOUR_COMPANY_WEBSITE]
Buyer segment: [BUYER_SEGMENT]
Offer: [COLD_OFFER]
Targets: [TARGET_1_DOMAIN], [TARGET_2_DOMAIN]
I have cloned agentic-gtm-workshop and opened it as this local project. Read README.md, AGENTS.md and START-HERE.md. If an input is blank, ask only for what official sources cannot supply. Suggest real targets for my confirmation if needed.
Use this workshop folder as my local project. Keep runtime in code/ and outputs in company-motion/. Run commands from code/.
Read my official website and save company-motion/COMPANY.md with sourced facts, the segment, offer, proof and actual next-step link. Ask only for missing facts. If the site blocks access, use official text I paste.
After company-motion/COMPANY.md exists, run from code/:
python3 -B gtm.py bootstrap --company-brief ../company-motion/COMPANY.md
Use its saved verified Python executable for later commands and standalone scripts.
If Python needs installation, name the official installer and stop there. Do not install Homebrew.
Read company-motion/AUTHORITY.md and company-motion/AGENTS.md. Keep outreach and spending paused.
Save both target domains and the actual bootstrap result in company-motion/SETUP.md. Append the completed step, actual file paths, checks and next task to company-motion/PROGRESS.md. Do not run later exercises.
Show the output and success check. Stop. Next: review your company brief.

You should have:

- COMPANY.md, SETUP.md, AUTHORITY.md and AGENTS.md.

Check before continuing:

- Setup found a supported Python version.
- Sending and spending are paused.
- The agent stops and names the next task.

If blocked: If web access is blocked, paste your official website text. If Python is missing, use the launcher's named official installer, then rerun this same step.

Next: review your company brief.

Check your company facts, buyers and offer

Reference: p1-company. Main workshop.

Open the brief. Correct the buyer and offer. Check every proof claim against its website source before your agent reuses it.

Time: 8 minutes. Difficulty: Easy.

Tools: Codex web access and your official website. No paid API required..

Do this:

1. Read COMPANY.md. Check the buyer, offer and next-step link.

2. Open its proof sources. Resolve conflicting numbers or leave them out.
3. Confirm the brief. Use it for both account pages.

Copy the complete prompt from [prompts.html#p1-company](#). Paste it into the same local Codex or Claude Code project.

Review `company-motion/COMPANY.md` with me.
Show the buyer segment, offer, next-step link and the source behind each proof claim.
If the website has conflicting numbers, show both URLs and exact excerpts. Do not choose a number without support.
Ask at most two short questions for missing company facts. Keep unsupported claims out of the brief.
Save my corrections and preserve the source URLs and read dates.
Append this step, actual artifact, check and next task to `company-motion/PROGRESS.md`. Show the output. Stop. Next: review the operating instructions.

You should have:

- `company-motion/COMPANY.md` with sourced company facts.

Check before continuing:

- Every proof claim has a supporting source.
- Buyer segment and offer match your answers.
- Conflicting website numbers are flagged, not silently selected.

If blocked: Paste text from your official product and proof pages. Codex saves the pasted source and labels it owner-provided.

Next: review the operating instructions.

Check the saved rules. Keep sending paused.

Reference: `p1-authority`. Main workshop.

Setup creates the operating instructions for you. Read the company facts and paused authority. Sending permissions are added in Part 2.

Time: 4 minutes. Difficulty: Easy.

Tools: Prepared bootstrap and your saved company brief. No API..

Do this:

1. Open `AUTHORITY.md` and `AGENTS.md`. Your local rules and sending limits.
2. Confirm sending and spending are paused.
3. Ask your agent to repair either missing file. Keep the existing company brief.

Copy the complete prompt from [prompts.html#p1-authority](#). Paste it into the same local Codex or Claude Code project.

Open company-motion/AUTHORITY.md and company-motion/AGENTS.md.
If either is missing, rerun:
`python3 -B gtm.py bootstrap --company-brief ../company-motion/COMPANY.md`
Preserve my existing facts, instructions and history.
Show the company name, one-step-at-a-time rule, and paused outreach and spending rules.
If either file conflicts with my company brief, show the conflict for my correction. Do not grant a sender or live channel yet.
Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop. Next: research the first target company.

You should have:

- Present, readable AUTHORITY.md and AGENTS.md.

Check before continuing:

- Both files exist.
- No outreach, sender, spend or country authority is invented.

If blocked: Rerun bootstrap with the existing company brief. Preserve the brief and working pages.

Next: research the first target company.

Chapter 3: Choose two target companies

Two target briefs and TARGETS.md.

Research your first target company

Reference: p1-one-source. Main workshop.

Your agent reads the target website. Keep one useful fact and its source beside the page idea it supports.

Time: 5 minutes. Difficulty: Easy.

Tools: Official target-company website. No paid API required..

Do this:

1. Choose the first target website. Use a real company you want to sell to.
2. Save three useful facts. Record its product, buyers and a relevant current need.
3. Mark unsupported ideas as suggestions. A proposed campaign is not a known business problem.

Copy the complete prompt from prompts.html#p1-one-source. Paste it into the same local Codex or Claude Code project.

Use the first target domain saved during setup.
Read its official website and my company brief.
Save company-motion/targets/ACTUAL-DOMAIN.md with its product, buyers and facts relevant to our offer. Use the actual domain in the path.
For each fact, include the source URL, exact supporting text and read date.
Keep ideas separate from facts. Do not invent revenue, spend, buying intent or a recent launch.
Show one sourced target fact and the page idea it supports.
Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop. Next: research the second target and save the target list.

You should have:

- One dated target brief.

Check before continuing:

- Each target fact has its exact source excerpt and read date.
- The page idea is labeled as a suggestion.

If blocked: Paste the target's official source text into Codex. Keep the official URL and retrieval date.

Next: research the second target and save the target list.

Save your two target companies

Reference: p1-target-fit. Main workshop.

These are companies you want to sell to. Save their domains and one sourced reason your offer fits each company.

Time: 5 minutes. Difficulty: Easy.

Tools: Two real target domains and browser access. No paid API required..

Do this:

1. Read the saved target domains. Use the two companies from setup.
2. Research the second company. Save its facts and one supported fit reason.
3. Save the two-company list. Keep uncertain fit visible for review.

Copy the complete prompt from prompts.html#p1-target-fit. Paste it into the same local Codex or Claude Code project.

Read the two target domains saved during setup.

Research the second company from its official sources and save its target brief.

Save company-motion/TARGETS.md with both actual domains, products, buyers and one sourced reason our offer fits each.

Keep uncertain fit marked for review. This list is page-building input, not permission to contact a person.

Show both companies plainly, one row each.

Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop. Next: build the first account page.

You should have:

- Two reviewed target briefs and company-motion/TARGETS.md.

Check before continuing:

- Each company has one supported reason for the page.

If blocked: Use two named companies you already know, then paste their official product pages.

Next: build the first account page.

Zuora: show useful ideas before asking for time

Reference: p1-page-example. Main workshop.

The page gives Zuora campaign ideas. It shows the offer and a clear next step. Your page should do the same for your buyer.

Chapter 4: Build and check your first page

First HTML page and PAGE-REVIEW.md.

Build the page for your first target

Reference: p1-build-page. Main workshop.

Use the first target's facts, your offer and supported proof. The page should make one relevant next step clear.

Time: 10 minutes. Difficulty: Easy.

Tools: Codex file/browser access and the two saved briefs. No API credits..

Do this:

1. Copy the page prompt. Codex reads the two saved briefs.
2. Open the generated page. View the HTML in your browser.
3. Check the headline and offer. They must refer to the correct seller and buyer.

Copy the complete prompt from prompts.html#p1-build-page. Paste it into the same local Codex or Claude Code project.

Build company-motion/pages/ACTUAL-DOMAIN/index.html for the first target using our company brief and its target brief.
Make one standalone responsive HTML file with styles, scripts and required graphics inside it.
Use our actual brand, one buyer-specific idea, our actual offer, supported proof and our real next-step link.
Keep suggestions distinct from facts. Do not invent a testimonial, logo, demo length, result or recent launch.
If proof is weak, use the supported product description or omit that proof block.
Save claim/source notes beside the page and open the actual file in a browser.
Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop. Next: check every claim and review desktop and phone layouts.

You should have:

- A real HTML page and source notes.

Check before continuing:

- The page opens.
- The seller, target and next step are correct.
- No unsupported proof appears.

If blocked: If browsing fails, use the saved official-source text. If preview is unavailable, open the generated file from its path.

Next: check every claim and review desktop and phone layouts.

Check every claim. Then check the page layout.

Reference: p1-page-review. Main workshop.

The page can look good and still make a false claim. Check every sentence against its source before approving the page.

Time: 10 minutes. Difficulty: Easy.

Tools: Browser and generated HTML. No paid API required..

Do this:

1. Read each factual claim. Open its exact source and supporting text.
2. Remove unsupported promises. Check demo length, numbers, testimonials and words such as "just launched".
3. Check desktop and phone. Measure the browser viewport, not the Mac window.
4. Open the next-step link. Repair failed checks and rerun them.

Copy the complete prompt from prompts.html#p1-page-review. Paste it into the same local Codex or Claude Code project.

Review every factual claim in the saved HTML, including headings, buttons, captions and footer. Save a claim ledger with the claim, exact source URL and supporting excerpt. Remove or rewrite unsupported demo length, results, testimonials and recency claims. Suggestions must be labeled. Check the page in actual browser viewports at 1440 x 900 and 390 x 844. Read window.innerWidth and window.innerHeight before checking overflow; resizing a Mac window alone is not proof of the viewport. If the browser cannot reach 390 pixels, record phone review as unverified and give a device-mode or phone check. Do not report false clipping. Test the next-step link. Repair failures and rerun them. Save company-motion/PAGE-REVIEW.md with the claim ledger, measured viewport sizes, screenshots and unresolved checks. Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop. Next: save the checked page method as a skill.

You should have:

- Reviewed page plus company-motion/PAGE-REVIEW.md.

Check before continuing:

- Every factual claim has exact supporting text or is removed.
- Measured innerWidth matches the recorded viewport.
- Phone overflow is measured inside that viewport.
- Failed checks are rerun after repair.

If blocked: If the agent cannot set a true phone viewport, record that check as unverified and use your browser's device mode or your phone. Do not label a Mac window-size limit as clipped page content.

Next: save the checked page method as a skill.

Chapter 5: Save the skill. Build page two.

SKILL.md, second page and completed file check.

Save your page-building instructions as a skill

Reference: p1-save-skill. Main workshop.

A skill is a file of instructions for a repeatable job. Save the method that produced your reviewed page.

Time: 6 minutes. Difficulty: Easy.

Tools: Reviewed first page and source notes. No API credits..

Do this:

1. Copy the skill prompt. Use your checked page and source notes.
2. Read the saved SKILL.md. It must say what to read, build and check.
3. Use these instructions for the next page.

Copy the complete prompt from prompts.html#p1-save-skill. Paste it into the same local Codex or Claude Code project.

Create company-motion/skills/account-page/SKILL.md from the method that passed review. Include the required company and target facts, standalone HTML output, source ledger, missing-proof rule, real next-step link and measured desktop/phone checks. Use the reviewed first page as the example. Make the instructions usable in a fresh local agent session. Require each factual claim to have exact source support. Keep unsupported facts out. Show the complete saved skill. Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop. Next: use that skill for the second target.

You should have:

- company-motion/skills/account-page/SKILL.md.

Check before continuing:

- The skill names inputs, output and checks.
- A missing source has a defined response.

If blocked: Use the complete skill prompt with the reviewed page and source notes. Save the skill in company-motion, then inspect it.

Next: use that skill for the second target.

Use the skill for your second target

Reference: p1-repeat. Main workshop.

Run the saved instructions for the second company. Check that its facts and page idea changed, and that the first page stayed intact.

Time: 8 minutes. Difficulty: Easy.

Tools: Saved account-page skill and second target brief. No paid API required..

Do this:

1. Run the account-page skill. Use the second saved target brief.
2. Open both pages. Compare buyer names and ideas.
3. Save the repeat result. Keep the first page intact.

Copy the complete prompt from prompts.html#p1-repeat. Paste it into the same local Codex or Claude Code project.

Use the saved account-page skill and the second target brief to build the second page.
Keep the first page unchanged.
Run the skill's full claim/source review and measured desktop/phone checks. A missing browser check stays unverified.
Show what changed between the two targets. Remove facts or generic copy carried over from the first company.
Save company-motion/SKILL-TEST.md with both page paths, review evidence and remaining issues.
Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop. Next: check the Part 1 files after the break.

You should have:

- Second reviewed page and company-motion/SKILL-TEST.md.

Check before continuing:

- Each page addresses its own buyer.
- The first page remains intact.
- The saved skill produced both outputs.

If blocked: If the second website blocks access, use its saved official-source text. Label that source path.

Next: check the Part 1 files after the break.

Save your pages. Take five minutes.

Reference: p1-break. Main workshop.

Save both pages. Leave the project open. Return in five minutes.

Follow the presenter. Keep your files open.

Check the files. Fix anything missing.

Reference: p1-checkpoint. Main workshop.

Your agent opens the required files. If setup skipped the instructions, bootstrap repairs them in the same project without losing your pages.

Time: 5 minutes. Difficulty: Easy.

Tools: Your generated files and actual browser results. No new credits..

Do this:

1. Copy the checkpoint prompt. Your agent opens each required file.

2. Fix the first failed check. Keep both pages and the saved skill.

3. Read PROGRESS.md. Check the result and your next task.

Copy the complete prompt from prompts.html#p1-checkpoint. Paste it into the same local Codex or Claude Code project.

Open our company brief, setup receipt, company-motion/AUTHORITY.md, company-motion/AGENTS.md, target list, two target briefs, two pages, page review, saved skill and skill test.

If company-motion/AUTHORITY.md or company-motion/AGENTS.md is missing, rerun bootstrap with the existing company brief. Preserve all working outputs and history.

Append actual paths and passed or missing checks to company-motion/PROGRESS.md.

Keep created, reviewed and published separate. Repair the first available local issue and rerun its check.

Name one exact next action for each remaining gap.

Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop. Next: finish the first remaining issue with individual help.

You should have:

- PROGRESS.md with actual paths and passed or missing checks.

Check before continuing:

- Required files are actually opened.
- Missing authority/instruction files can be repaired in place.
- Created, reviewed and published remain separate labels.

If blocked: Share the failed file or error with the repair prompt. Continue from the same project.

Next: finish the first remaining issue with individual help.

Fix your first unfinished task

Reference: p1-help. Main workshop.

Work individually. Open the repair prompt for the failed task. Gil helps with errors and unclear results.

Time: 30 minutes. Difficulty: Help.

Tools: Codex and your workshop project..

Do this:

1. Open PROGRESS.md. Find the first unfinished check.
2. Copy that task's repair prompt from prompts.html.
3. Add your error. Rerun the check. Save the result.

Copy the complete prompt from prompts.html#p1-help. Paste it into the same local Codex or Claude Code project.

Read company-motion/PROGRESS.md and find the first unfinished Part 1 check.
Repair that local cause in this same project. Preserve working pages, source notes and the saved skill.
Rerun its success check and update company-motion/PROGRESS.md.
If only phone review is missing, show the exact manual device-mode or phone check instead of inventing a pass.
Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop. Next: prepare the single email connection before Part 2.

You should have:

- One repaired task and updated progress.

Check before continuing:

- The previously failed check is rerun.

If blocked: Save the unresolved error and named missing setup. Do not reset the project.

Next: prepare the single email connection before Part 2.

Register for Parts 2 and 3. Prepare your email.

Reference: p1-before-part2. Optional extension.

After class, follow PREWORK.md for the next workshop. Keep using the same project.

Optional after class. Difficulty: Setup.

Tools: Google sender/calendar and ZeroBounce for the live email test. Finder optional..

Do this:

Copy the complete prompt from prompts.html#p1-before-part2. Paste it into the same local Codex or Claude Code project.

Read PREWORK.md, API-CONNECTIONS.md and SETUP-SEQUENCE.md for the single email channel.
Prepare my own sender, primary calendar, permitted countries and current customer/deal/opt-out exclusions.
Follow the Google Cloud sign-in guide. It must initialize this fresh email configuration from code/ using python3 -B gtm.py init --config and the actual private path before treating email setup as ready; bootstrap alone is local file setup. Use the project launcher and the actual private client-file path.
Record the authenticated sender and calendar; name any Workspace-admin blocker.
Prepare ten distinct target domains and current buyer contacts plus my own self-test address.
Read the actual ZeroBounce balance and show the validation cost. Ask for my bounded paid-validation approval before spending credits. Keep original pending request IDs.
Save company-motion/CONNECTION-CHECK.md with each actual result or blocker. Keep outreach paused.
Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop. Next: open Part 2 and check your email prerequisites.

You should have:

- CONNECTION-CHECK.md with ready or blocked email prerequisites.

Check before continuing:

- Own Google profile and calendar reads succeed.
- Actual ZeroBounce credits and paid validation scope are known.

- Current exclusions and permitted countries are ready.

If blocked: Without sender authorization, Part 2 can prepare messages. It cannot prove live sending.

Next: open Part 2 and check your email prerequisites.

Part 2: Adapt the workflow. Test one email channel.



Finish with

- Local preparation: execution flow, source-backed ten-company universe and exact message previews.
- When email is connected: an actual tested exclusion and current buyer/validation evidence.
- When release is authorized: one exact self-test Sent match before any prospect send.
- Conditional prospect pilot: actual sent, held or unknown result for every original key.
- Dated gaps and next actions when live authorization or evidence is missing.

Before this session

- Bring the same project and reviewed company brief.
- Complete own Google sender and primary-calendar authorization before class.
- Have current customer, deal, opt-out and relationship exclusions.
- Have permitted company countries, ten distinct targets and eleven exact current validated email addresses.
- Use ZeroBounce credits only within your approved validation scope. A paid finder is optional.

Five chapters

- 1. Connect your email account: Verified owned sender, calendar, countries and current exclusions.
- 2. Set the rules and choose ten companies: Execution flow, exclusion test and ten-company universe.
- 3. Find buyers and write the emails: Current buyer evidence and source-checked email copy.
- 4. Review and send your own test: Exact approved messages and own-sender GmailSent proof.
- 5. Run the ten-company pilot: Each original result, labeled sent, held or unknown.

Chapter 1: Connect your email account

Verified owned sender, calendar, countries and current exclusions.

Your page skill becomes part of a workflow

Reference: p2-start. Main workshop.

Your pages and saved skill are ready. Today you connect one email channel, prepare ten target companies and inspect one real self-test before sending to prospects.

Check your email connection before sending

Reference: p2-connect. Main workshop.

Initialize your email configuration, then read the sender, calendar, countries and exclusions. Missing people or message holds are expected here; you prepare those next.

Time: 12 minutes. Difficulty: Easy.

Tools: Google OAuth, current exclusions and participant config. Read-only calls..

Do this:

1. Initialize once for email operation. Bootstrap created local files. Init loads your own private email configuration.
2. Initialize your email configuration. The agent runs init once with your private config. Later deliberate changes use configure.
3. Read the four connection checks. Owned sender, calendar, country scope and current exclusions.
4. Separate connection failures from later holds. People and messages are built in the next exercises.

Copy the complete prompt from prompts.html#p2-connect. Paste it into the same local Codex or Claude Code project.

Read my company brief, authority, company-motion/CONNECTION-CHECK.md and the email setup guide.

Ask only for missing sender or permitted-company-country choices. Show the saved canonical countries; US and United States must match. Hold unknown countries.

Create or update my private configuration through the package's exact schema. Keep the eleven-email ceiling, one owned self-test, ten prospect limit and other live channels disabled.

Use snapshot mode for this owner-reviewed manual test. Bootstrap alone does not initialize email operation. From code/, run `python3 -B gtm.py init --config` with the actual absolute private-config path once for this fresh project. Verify `initialized:true`. Use `configure` only for later deliberate configuration changes; preserve saved grants and history.

Read my actual Google profile and primary calendar. Re-read current customer, deal, opt-out and relationship sources; save and import complete current exclusions with their real read time and owner review.

Run ready and save its actual output in company-motion/CONNECTION-CHECK.md. Mark this connection step passed only when the owned profile/calendar, canonical permitted countries and complete current exclusions pass. Missing people/actions are expected until later exercises; record those holds without restarting setup. Bookings expire after 25 minutes, history after 60 and owner exclusions after 65; stale or partial evidence holds live release.

Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop. Next: review and save the order of work.

You should have:

- CONNECTION-CHECK.md with connection results and expected later recipient/action holds.

Check before continuing:

- Init loads the fresh private configuration and returns initialized:true.
- Owned Google profile/calendar, canonical permitted countries and current complete exclusions pass.
- Missing people/actions remain expected later holds, not a reason to reset setup.

If blocked: Without OAuth or complete exclusions, prepare messages and record the missing setup. Live proof remains incomplete.

Next: review and save the order of work.

A key does not finish a connection

Reference: p2-key-vs-account. Main workshop.

An API key opens an account. You still need the right sender, current recipients, exact messages and permission to send. Check those separately.

Chapter 2: Set the rules and choose ten companies

Execution flow, exclusion test and ten-company universe.

STEP 2: adapt the prepared order of work

Reference: p2-flow. Main workshop.

Keep the prepared checks. Save your company's offer and the next action for each result. Your agent prepares; the local program checks and sends.

Time: 7 minutes. Difficulty: Easy.

Tools: Company brief, authority and current database. No new paid call..

Do this:

1. Write the next-action rules. Use your company, exclusions and supported channels.
2. Save the operating instructions. Keep selection and sending responsibilities explicit.
3. Trace one target through the rules. Show the expected next action without sending.

Copy the complete prompt from prompts.html#p2-flow. Paste it into the same local Codex or Claude Code project.

Save company-motion/EXECUTION-FLOW.md beside my company brief and update company-motion/AGENTS.md.

Use this order: read history and exclusions; select current buyers; write exact sourced messages; inspect and approve the eleven-message cohort; refresh checks; send my own self-test; match it in Gmail Sent; release only approved prospect keys; read results, replies and bookings; save one next action.

Name the local agent as researcher/writer and gtm.py as checker/sender/record keeper.

Keep uncertain provider results held for original-ID readback. Show the next action for one actual target. Keep sending paused.

Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop. Next: test one real excluded company.

You should have:

- company-motion/EXECUTION-FLOW.md and company-motion/AGENTS.md.

Check before continuing:

- The writer and sender have distinct tasks.
- An empty action list causes preparation, not repeated empty runs.

If blocked: Use the complete flow prompt and the actual runtime AGENTS.md as reference. Save your company's operating instructions separately.

Next: test one real excluded company.

Test one company you do not want contacted

Reference: p2-rule-test. Main workshop.

Choose a real company outside the pilot. Add its exclusion and run a read-only rule check. The actual decision must say hold.

Time: 5 minutes. Difficulty: Easy.

Tools: Current target and exclusions import/review. No paid call..

Do this:

1. Choose a real company outside the pilot. Use its actual domain and your explicit exclusion.
2. Import the local exclusion. Keep prior exclusions and the true source timestamp.
3. Run the prepared rule-test command. The agent reads the actual excluded domain. No helper script is needed.
4. Read the hold reason. No email action or provider write is created.

Copy the complete prompt from prompts.html#p2-rule-test. Paste it into the same local Codex or Claude Code project.

Choose one real company outside the ten-prospect pilot that I want excluded.
 Save and import its actual company facts and my domain exclusion. Preserve earlier exclusions.
 Read the actual saved domain from those records; do not ask me to transcribe it.
 From code/, run `python3 -B gtm.py rule-test --domain` with that actual domain.
 Use this prepared read-only command. Do not generate a database helper script, create a person or prepare an action.
 Save the actual decision, matching exclusion and unchanged action/attempt counts in company-motion/RULE-TEST.md. It must be HOLD for our excluded domain. NOT_TESTED is a missing check, not success. Record that this domain-only test did not validate any person's email.
 Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop. Next: build your ten-company target list.

You should have:

- RULE-TEST.md with actual HOLD, exclusion and unchanged action/attempt counts.

Check before continuing:

- Prepared rule-test returns HOLD for the actual excluded domain.
- No action, attempt or provider write is created.
- NOT_TESTED is recorded as incomplete.

If blocked: If the command returns NOT_TESTED, repair its named missing account or exclusion. Keep the incomplete check visible.

Next: build your ten-company target list.

STEP 3: save ten companies you want to sell to

Reference: p2-universe. Main workshop.

Use your existing list or ask the agent to find matches. Keep the actual domain, source, country and fit reason for every company.

Time: 10 minutes. Difficulty: Easy.

Tools: Ten target companies, official-source web access or one funded research provider..

Do this:

1. Use ten real target companies. Import an existing list or let Codex find suitable companies.
2. Read each company's official source. Save the fit reason and current domain.
3. Check exclusions. Keep excluded companies visible with a reason.

Copy the complete prompt from prompts.html#p2-universe. Paste it into the same local Codex or Claude Code project.

Use my supplied target list or find ten distinct real companies from the saved buyer segment and permitted countries.
Keep the two Part 1 targets when they fit; do not include the company used for the exclusion test.
For every company, save domain, name, actual country, one fit reason, source URL, supporting excerpt and read date.
Normalize supported country aliases through the package. Hold unknown country or fit rather than guessing.
Write company-motion/ACCOUNTS.json with the exact package schema, import accounts and read each saved account.
Save a ten-row list with included, held or excluded and one plain reason. Do not contact anyone.
Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop. Next: check one sourced reason to contact a target.

You should have:

- ACCOUNTS.json and ten-row company fit/exclusion list.

Check before continuing:

- Ten distinct real companies are present.
- Domains and fit reasons have current support.
- Excluded accounts are not eligible.

If blocked: Without a starting list, let Codex find ten distinct real companies from official web sources and the saved buyer criteria. Paid finding is optional.

Next: check one sourced reason to contact a target.

A segment is a group of target companies

Reference: p2-segments. Main workshop.

For example, security software companies selling to large businesses. Each attendee uses their own buyer segment. This is individual work.

Use a sourced reason, not a guessed buying signal

Reference: p2-signal. Main workshop.

Choose one target fact that makes your offer relevant. A dated announcement can help. A public comment alone does not prove the person wants to buy.

Time: 5 minutes. Difficulty: Easy.

Tools: Official web research. No paid API required for this class step..

Do this:

1. Read one current official event or public post. Save the source, date and exact text.
2. Check why it matters to your offer. Keep the reason specific.
3. Save the next action. Research or prepare only what the evidence supports.

Copy the complete prompt from [prompts.html#p2-signal](#). Paste it into the same local Codex or Claude Code project.

Choose one target from our saved list.

Use an official announcement, product page or relevant public post to explain why our actual offer may help.

Save company-motion/SIGNAL.md with the exact URL, supporting excerpt, actual date and matching company or person. Keep facts separate from our inference.

Do not turn an undated page into a recent launch or a public comment into purchase intent.

Use official web research for this class step. An optional paid finder or Apify run needs its own guide, bounded cost approval and original-run readback.

Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop. Next: check the current buyers and exact email addresses.

You should have:

- One sourced dated signal and next action.

Check before continuing:

- The quote exists in the provider source.
- The identity and date match.
- Pending paid work keeps its original handle.

If blocked: Use a current official announcement from the company's website. Mark paid signal scanning as not connected.

Next: check the current buyers and exact email addresses.

Chapter 3: Find buyers and write the emails

Current buyer evidence and source-checked email copy.

Check who works there and the exact email address

Reference: p2-buyer. Main workshop.

Current employer evidence and email validation are separate. Use the existing contacts first. Pay for a finder only where facts are missing.

Time: 10 minutes. Difficulty: Easy.

Tools: Current buyer sources; one finder if needed; ZeroBounce credits for exact email validation..

Do this:

1. Open the ten prepared people. Match each person to the actual employer.
2. Repair missing evidence. Use one connected finder only when needed.
3. Validate each exact address. Check balance and enable paid research first.
4. Inspect one imported row. Employer and email evidence remain separate.

Copy the complete prompt from prompts.html#p2-buyer. Paste it into the same local Codex or Claude Code project.

Read the exact people schema and our ten company records.

Use current buyer contacts prepared before class. Repair missing role/employer evidence from official sources or an approved finder.

Read the actual ZeroBounce balance. Before paid validation, show the exact addresses and cost scope for my approval. Follow the guide's research-on contract and preserve any existing SixtyFour ceiling. Validate each released address and my own self-test address. Keep its actual receipt separate from current-employer evidence; both must be within seven days.

Preserve original pending request IDs. Keep invalid, catch-all, guessed or unknown addresses held.

Write and import company-motion/PEOPLE.json using the exact schema. Show one full evidence row and the remaining held rows. Turn paid research off after the approved work.

Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop. Next: write and source-check the exact first emails.

You should have:

- Current person records and separate provider email-validation receipts.

Check before continuing:

- Person works at the intended company.
- Exact email result meets the send check.
- No name or email pattern is guessed.

If blocked: Use contacts you already know and current official evidence. Without email-validation access, save them held and complete validation before live sending.

Next: write and source-check the exact first emails.

The real Forum One email

Reference: p2-real-email. Main workshop.

"May I send you campaign ideas for reaching prospective Forum One clients?" The email names what Forum One sells before making this ask.

Write the email from facts you can prove

Reference: p2-email. Main workshop.

Use one relevant fact, your actual offer and one permission ask. Show the exact recipient and message. Keep the first email free of links.

Time: 10 minutes. Difficulty: Easy.

Tools: Company and person evidence. Codex account only; no sending call..

Do this:

1. Read the company and buyer facts. Use one supported reason to contact the buyer.
2. Generate the exact message. Show sender, recipient, subject, plain text and HTML.
3. Check every factual claim. Remove unsupported promises and keep the first email link-free.
4. Save all exact previews. Do not import or send before inspection.

Copy the complete prompt from prompts.html#p2-email. Paste it into the same local Codex or Claude Code project.

Read our company brief, flow rules and current buyer evidence.
Write one first email for each eligible prospect: one supported relevance reason, our real offer and one permission ask. Keep it link-free.
Check every factual claim against its source. Do not invent a demo length, results or a recent launch. Omit weak unsupported proof.
Use my actual approved sender, signature and opt-out instruction from company-motion/AUTHORITY.md. Ask for any missing sender fact.
Save matching plain text and HTML in company-motion/EMAIL-PREVIEWS.md and company-motion/PROSPECT-DRAFTS.json with exact package fields.
Show every recipient, subject and body. Do not import or send until I inspect them.
Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop. Next: review all eleven messages and pin exact approval.

You should have:

- EMAIL-PREVIEWS.md and ten exact prospect drafts.

Check before continuing:

- Each message has one real relevance reason.
- The first email has no link.
- Recipient, sender, subject and body are visible.

If blocked: If evidence is weak, use a simple conditional sentence about the company's offer. Keep unsupported claims out.

Next: review all eleven messages and pin exact approval.

Chapter 4: Review and send your own test

Exact approved messages and own-sender GmailSent proof.

Review your self-test and ten prospect emails

Reference: p2-prepare-ten. Main workshop.

One message goes to your own sender. Ten go to distinct target companies. Inspect each exact message, then save approval against its original key.

Time: 8 minutes. Difficulty: Easy.

Tools: Ten company/person records and current evidence. Codex only until live approval..

Do this:

1. Prepare the self-test and ten prospects. Use the current imported people and exact copy.
2. Inspect all eleven rows. Check sender, recipient, subject and both bodies.
3. Save the exact action keys. Approval pins content and the eleven-email lifetime cap.
4. Keep execution paused. The self-test must pass before prospect release.

Copy the complete prompt from `prompts.html#p2-prepare-ten`. Paste it into the same local Codex or Claude Code project.

Prepare one owned self-email and ten distinct prospect-company emails from our reviewed drafts. Follow the exact controlled-test and pilot schema in the guide. My self-test sender equals recipient, is allowed in `controlled_test_recipients`, and both person and payload use `controlled_test:true`. It still needs current evidence and validation.

Save `company-motion/PILOT-ACTIONS.json` and an eleven-row `company-motion/PILOT-REVIEW.md` showing sender, recipient, subject, plain text, HTML and holds.

Import each action once. Show all eleven original keys and exact messages.

After my explicit approval, write `company-motion/PILOT-APPROVAL.json` with that approval, the original `controlled_action_key` and ten original `prospect_action_keys`. Run `approve-pilot` and save its receipt.

Keep sending paused. The eleven-email limit lasts across days. Changed approved copy or recipient stays held; preserve the grant and use the separate correction procedure.

Append this step, actual artifact, check and next task to `company-motion/PROGRESS.md`. Show the output. Stop. Next: send only your approved self-test after the break.

You should have:

- Eleven-message review, original action keys and exact approval receipt.

Check before continuing:

- One self-test and ten distinct prospect company domains are present.
- Every ready row has current identity, validation and exclusions.
- Owner approval pins the exact eleven immutable action keys.
- No approval is inferred from a draft or mutable file.

If blocked: Use the help period to repair missing rows. A smaller ready set is a partial pilot.

Next: send only your approved self-test after the break.

Approval names the exact eleven emails

Reference: `p2-live-approval`. Main workshop.

The approval receipt pins all eleven action keys. Send your self-test first. Ten prospect keys become releasable after its exact Gmail Sent match.

Five minutes. Keep the message open.

Reference: `p2-break`. Main workshop.

Save the exact email preview. Return with the same sender and recipient visible.

Follow the presenter. Keep your files open.

Send only the approved email to yourself

Reference: p2-canary. Main workshop.

Refresh the checks. Release the original self-test key once, then pause. If its result is uncertain, read the original attempt instead of sending again.

Time: 6 minutes. Difficulty: Moderate.

Tools: Own connected Google sender; current owner identity and source checks. One live self-email..

Do this:

1. Read the pinned eleven-key approval. All messages were already imported and reviewed.
2. Inspect the approved self-test. Sender and recipient are your authorized mailbox.
3. Release only that self-test. Select its original approved action key. The ten prospect keys remain held until exact Sent.
4. Run once, read the result and pause. Keep the original key if its outcome is unknown.

Copy the complete prompt from `prompts.html#p2-canary`. Paste it into the same local Codex or Claude Code project.

Read the successful original pilot approval and exact self-test preview. Do not import actions again. Refresh current sources and run review. Reread real exclusions if stale; do not change timestamps. Confirm only the self-test can pass before prospect release.

Read the original self-test key from the saved approved manifest; I do not transcribe keys. After I approve releasing it, construct the exact run command with `--limit 1` and `--action-key` set to that original key. Show the selector, run once, then pause.

Save `company-motion/CANARY.md` with that key and actual provider result. Reconcile uncertain results through the original key. The manual bounded command can run outside scheduled hours but keeps all recipient checks and caps.

Append this step, actual artifact, check and next task to `company-motion/PROGRESS.md`. Show the output. Stop. Next: match the actual self-test in Gmail Sent.

You should have:

- One attempted test email with exact action key and provider result.

Check before continuing:

- Exact approved self-test is the only first release.
- Prospects require its exact Gmail Sent match.
- Self-test counts toward eleven lifetime emails but not ten prospect results.
- Unknown outcomes cause readback rather than a resend.
- The manual command selects the original self-test key explicitly.

If blocked: Without an authorized sender, keep the exact test email prepared. Live replication remains incomplete.

Next: match the actual self-test in Gmail Sent.

Open the actual message in Gmail Sent

Reference: p2-sent. Main workshop.

Compare the sent message with the approved preview. A response ID alone does not prove that exact message was sent. A Sent match does not prove delivery.

Time: 5 minutes. Difficulty: Easy.

Tools: Gmail readback and the controlled self-test key. Read-only checks..

Do this:

1. Read the original test attempt. Use its saved action key and provider ID.
2. Match the exact Sent message. Check sender, recipient, subject, plain text and HTML.
3. Keep its original history. Inspect the attempt without importing or releasing another action.

Copy the complete prompt from prompts.html#p2-sent. Paste it into the same local Codex or Claude Code project.

Reconcile the original self-test action key.

Open its exact Gmail Sent message. Compare sender, recipient, subject, plain text and HTML with the approved preview.

Save the native message ID, exact match and observed stage in company-motion/CANARY.md.

Inspect the original attempt and history with results and person reads.

Keep execution paused. Do not reimport or resubmit for a duplicate test.

If the exact Sent match is missing, keep the ten prospect messages held and read that original attempt again later.

Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop. Next: release only the approved prospect cohort after the self-test passes.

You should have:

- One exact Gmail Sent match and its unchanged original attempt record.

Check before continuing:

- Exact Sent content matches the pinned message.
- The original key, provider ID and attempt record remain intact.
- No additional import or release was used for this check.

If blocked: If provider Sent cannot be confirmed, keep the state uncertain. Reconcile the existing attempt later.

Next: release only the approved prospect cohort after the self-test passes.

The test email is not one of the ten prospects

Reference: p2-counts. Main workshop.

The workshop email limit is eleven: one controlled test email plus ten distinct prospects. Count only exact prospect Sent matches toward the pilot.

Chapter 5: Run the ten-company pilot

Each original result, labeled sent, held or unknown.

STEP 4: test emails for ten target companies

Reference: p2-pilot. Main workshop.

After the exact self-test passes, release only the ten original prospect keys you approved. Held rows stay held. Pause and inspect every result.

Time: 3 minutes. Difficulty: Moderate.

Tools: Google sender and approved ten-account file. Up to ten live prospect emails..

Do this:

1. Read the existing pinned approval. Use its ten prospect keys and successful self-test receipt.
2. Refresh and inspect current checks. Do not import or replace the approved actions.
3. Release the ten pinned prospect keys. Select its ten original action keys explicitly. Run once, then pause.
4. Read each original result. Keep Sent, held and unknown separate.

Copy the complete prompt from `prompts.html#p2-pilot`. Paste it into the same local Codex or Claude Code project.

Read the original pilot approval and exact self-test Gmail Sent match.

Use only its ten original approved prospect keys. Do not reimport, swap held recipients or create a new pilot.

Reread current exclusions when required, then refresh and review. Stale, missing or partial evidence holds release.

Show the exact eligible messages and obtain my approval to release this cohort.

Read the ten original keys from the saved approved manifest; I do not transcribe them. Construct the bounded command with `--limit 10` and one `--action-key` selector for each key. Show that it excludes the self-test and other grants. Resume, run once and pause.

Reconcile each original result and save `company-motion/PILOT-RESULTS.md` with prepared, held, accepted, exact Sent or unknown per company. Do not call Sent delivery or a meeting.

Preserve the eleven-email lifetime ceiling and all earlier attempts.

Append this step, actual artifact, check and next task to `company-motion/PROGRESS.md`. Show the output. Stop. Next: read the result and next action for each company.

You should have:

- Actual result for each prospect and `company-motion/PILOT-RESULTS.md`.

Check before continuing:

- Live actions stay within approval and the eleven-email workshop cap.
- Ten exact prospect Sent matches are required for a full pilot.
- Fewer matches are labeled partial.
- Repeated action-key flags select only the ten original pinned prospect keys.

If blocked: Provider waits continue after class. Keep the same keys and reconcile them. Do not repeat uncertain writes.

Next: read the result and next action for each company.

Save the result for each company

Reference: p2-results. Main workshop.

Count exact Sent matches separately from held and unknown results. Save the next task for each company from its actual result.

Time: 2 minutes. Difficulty: Easy.

Tools: Codex and your workshop project..

Do this:

1. Read results and the pinned grant. Count ten distinct prospect domains separately from the self-test.
2. Inspect each original attempt. Use results or person with the actual recipient.
3. Save unresolved work. Keep original IDs and next reads in WAITS.md.

Copy the complete prompt from prompts.html#p2-results. Paste it into the same local Codex or Claude Code project.

Read the original pilot results and saved attempt history. Use read-only results, person and original-key reconciliation where supported.

Save a dated row per company: original key, exact provider stage, evidence and one next action. Count held, unknown and exact Sent separately. Full ten-prospect sending needs ten exact Sent matches. Partial results remain partial.

Do not reimport or release the cohort to prove repeat protection. Check its saved reservations and attempts without another write.

Update company-motion/PROGRESS.md and company-motion/WAITS.md. Keep sending paused.

Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop. Next: repair the first held prerequisite with individual help.

You should have:

- Dated pilot result and next actions.

Check before continuing:

- Denominator is ten distinct prospect domains.
- Self-test stays separate.
- Each stage has its original key and provider record.
- No pending result is labeled sent.
- No action was reimported or released during inspection.

If blocked: Keep unresolved rows in the progress record until provider evidence resolves them.

Next: repair the first held prerequisite with individual help.

Finish the ten-account preparation

Reference: p2-help. Main workshop.

Work individually. Fix the first incomplete prospect. Gil helps with failed connections, sources and message checks.

Time: 30 minutes. Difficulty: Help.

Tools: Codex and your workshop project..

Do this:

Copy the complete prompt from [prompts.html#p2-help](#). Paste it into the same local Codex or Claude Code project.

Read `company-motion/PILOT-REVIEW.md`, `company-motion/PILOT-RESULTS.md` and the actual held reason.

Repair the first available prerequisite while preserving original keys, approved copy, grants, unknown outcomes and limits. Rerun its failed read/check.

If an approved recipient or message must change, keep the original held and use the guide's separately reviewed correction procedure. Do not reset the pilot or its count.

If account access or owner approval is missing, name that one blocker. Update the result and next action.

Append this step, actual artifact, check and next task to `company-motion/PROGRESS.md`. Show the output. Stop. Next: bring the same project and results to Part 3.

You should have:

- Reviewed pilot file with repaired rows.

Check before continuing:

- Every ready row has actual supporting evidence.
- Changed approved copy cannot reuse or reset the original pilot grant.

If blocked: Keep a changed approved action held. Owner-approved revoke-grant permanently stops that grant without restoring or resetting the pilot. A replacement needs separate exact review.

Next: bring the same project and results to Part 3.

A manual test runs now. The limits still apply.

Reference: `p2-force`. Main workshop.

The workshop's manual test can run outside scheduled hours. It still checks your approved messages, recipients and send limits.

Before Part 3: bring the same project

Reference: `p2-before-part3`. Main workshop.

Bring your pages, messages and saved results. We will check replies, bookings and repeated operation. Extra channels are optional and require their own accounts.

Time: 0 minutes. Difficulty: Setup.

Tools: Same project and own connected email/calendar. No new required paid channel..

Do this:

Copy the complete prompt from [prompts.html#p2-before-part3](#). Paste it into the same local Codex or Claude Code project.

Read `company-motion/PROGRESS.md`, `company-motion/PILOT-RESULTS.md` and the setup guide. Verify that I can reopen this same project and read its original saved results. For the main Part 3 lane, keep the existing email/calendar connection and current exclusion sources. Do not require every extra channel. If I choose an extension, name its actual account, resource, cost and approval prerequisites from `API-CONNECTIONS.md`. Keep unfinished provider results with their original IDs in `company-motion/WAITS.md` and leave new sending paused. Append this step, actual artifact, check and next task to `company-motion/PROGRESS.md`. Show the output. Stop. Next: open Part 3 and continue the saved workflow.

You should have:

- Updated progress and one chosen extension only if ready.

Check before continuing:

- The same saved project and history remain intact.
- Optional channel setup is distinct from the main lane.

If blocked: A missing channel stays incomplete. Other tested channels can continue.

Next: open Part 3 and continue the saved workflow.

Part 3: Operate your working play. Add channels when ready.



Finish with

- Core local framework, two reviewed pages, reused skill, flow and prepared target list.
- Conditional reply/booking reads and manual provider cycles when connected.
- Retained history and verified pause/recovery when the runtime is initialized.
- Optional schedule stays prepared-paused until its actual gates and approval pass.
- A completion record separating local outputs, conditional results and optional gaps.

Before this session

- Bring the same project, reviewed pages, flow rules and Part 2 results.
- Keep own sender/calendar and current exclusion sources available.
- No new paid channel is required for the main lane.
- Optional social, gifting, hosting, ads and unattended operation require the guide's specific owned resources and approvals.

Five chapters

- 1. Read replies and bookings: Actual response states, or a clear missing-connection record.
- 2. Check one complete play: Evidence from company facts through reviewed output and actual result.
- 3. Repeat without duplicating work: Two bounded cycles, retained history and pause/reopen proof.
- 4. Prepare a schedule when ready: Readiness record. Schedule remains paused unless every gate passes.
- 5. Record results and choose the next play: Honest completion record. Extra channels are optional guides.

Chapter 1: Read replies and bookings

Actual response states, or a clear missing-connection record.

Keep the useful play working

Reference: p3-start. Main workshop.

Your project has company facts, skills, target companies and saved actions. Today you read what happened and choose the next step without repeating old actions.

Read replies before any cold follow-up

Reference: p3-replies. Main workshop.

Read the actual thread. Stop cold follow-ups after a reply. Draft the next response for review. An empty inbox is an honest result.

Time: 12 minutes. Difficulty: Easy.

Tools: Connected own Gmail and scoped thread history. A real reply is optional; drafting is local..

Do this:

1. Run the reply read. Use the sender's actual native thread.
2. Inspect the classification. Separate human, automatic, bounce and opt-out.
3. Prepare the next answer. Keep it in the existing conversation.
4. Save the stop decision. Cold outreach must not conflict with the reply.

Copy the complete prompt from prompts.html#p3-replies. Paste it into the same local Codex or Claude Code project.

If the required connection or initialized runtime is unavailable, save that exact gap in company-motion/REPLIES.md and name the next setup task. Do not claim the read or cycle completed; keep the finished local page play intact. Show the gap and next setup task, then stop here. Otherwise continue below.

Refresh and read the actual scoped mailbox/thread history for our original test and pilot.

Save company-motion/REPLIES.md with each exact thread ID, observed reply or no-reply result and next action.

Confirm any reply, opt-out or relationship excludes further cold follow-up. Use the current runtime's actual checks.

For a real reply, draft a short response from its words and our sourced offer. Keep it local for review; automatic reply sending is not wired.

If no reply exists, say so. Use a clearly labeled local fixture only to inspect stop logic. Never count that fixture as a real response.

Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop. Next: read actual bookings and cancellations.

You should have:

- Native reply classification, stop decision and reply draft where useful.

Check before continuing:

- The reply is bound to the exact earlier thread.
- Opt-outs and human replies exclude cold outreach.
- Drafting is not counted as sending.

If blocked: If the pilot has no reply, read an existing permitted provider conversation. Label its source. A missing actual reply remains incomplete.

Next: read actual bookings and cancellations.

Read the actual booking record

Reference: p3-bookings. Main workshop.

A booking changes the next action. Read the calendar or Calendly record. Keep cancellation, rescheduling and attendance separate.

Time: 10 minutes. Difficulty: Easy.

Tools: Google Calendar. Calendly token and organization/event scope only if used..

Do this:

1. Read the configured booking source. Use your own organization and event scope.
2. Match the attendee and company. Save the actual native meeting ID.
3. Check a reschedule or cancellation. Keep the replacement meeting active.
4. Inspect the cold-outreach exclusion. An active booking changes the next action.

Copy the complete prompt from prompts.html#p3-bookings. Paste it into the same local Codex or Claude Code project.

If the required connection or initialized runtime is unavailable, save that exact gap in company-motion/BOOKINGS.md and name the next setup task. Do not claim the read or cycle completed; keep the finished local page play intact. Show the gap and next setup task, then stop here. Otherwise continue below.

Read bookings from my configured owned calendar or Calendly source.

Save company-motion/BOOKINGS.md with actual source IDs, start times and current states. A successful empty read is zero bookings, not a connection failure.

For an actual reschedule, preserve the old record and active replacement. Verify an active booking holds cold follow-ups.

If I want a controlled calendar test, show the exact owned test event for approval before creating it, label it as a test and remove it only with my approval.

Do not infer attendance from a booking. Automatic attendance attribution is not wired.

Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop. Next: inspect your complete working play.

You should have:

- BOOKINGS.md with actual booking states or a successful empty read.

Check before continuing:

- Actual booking IDs and state are saved when present.
- An empty read and attendance are labeled separately.

If blocked: Use an existing permitted meeting or a willing external test contact's invitation. Label a controlled meeting as a test.

Next: inspect your complete working play.

Chapter 2: Check one complete play

Evidence from company facts through reviewed output and actual result.

Show your first working play from start to finish

Reference: p3-working-play. Main workshop.

Open its target brief, exact output and actual result together. Check every handoff. Save the next action and the one missing piece, if any.

Time: 12 minutes. Difficulty: Easy.

Tools: Saved company/target files and actual provider records. No new channel..

Do this:

1. Open one completed play. Read its target facts, output and review together.
2. Trace its actual result. Show file evidence or the exact provider record.
3. Repair one broken local handoff. Preserve committed actions and rerun the check.

Copy the complete prompt from prompts.html#p3-working-play. Paste it into the same local Codex or Claude Code project.

Choose our completed account-page or email play.
Open the actual target brief, saved skill, reviewed output and result together.
For a page, trace target fact to page claim to source review. For email, trace current buyer evidence to exact approved message to original attempt to Gmail Sent match.
Repair one unsupported claim or broken handoff and rerun its check without replacing a committed action.
Save company-motion/WORKING-PLAY.md with actual paths/IDs, passed checks, exact stage and one next action. Label local-only and live outcomes separately.
Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop. Next: take the five-minute break.

You should have:

- WORKING-PLAY.md with a complete evidence chain.

Check before continuing:

- Each handoff has an actual file or provider ID.
- Unsupported claims and missing stages remain visible.

If blocked: Use the missing operation as a concrete implementation task for your team or Metadata.

Next: take the five-minute break.

Five minutes. Save the channel results.

Reference: p3-break. Main workshop.

Save your working-play record. Leave the same project open. Return in five minutes.

Follow the presenter. Keep your files open.

Chapter 3: Repeat without duplicating work

Two bounded cycles, retained history and pause/reopen proof.

Run two preparation cycles without repeating actions

Reference: p3-manual-cycles. Main workshop.

The second cycle must read the first cycle's history. It may prepare a new draft. It cannot resend an earlier action or grant itself new permission.

Time: 12 minutes. Difficulty: Easy.

Tools: Same project, supported reads and existing exact grants. No new send required..

Do this:

1. Run a paused preparation cycle. Read history and current sources before choosing work.
2. Repeat from the same project. The second cycle reads the first record.
3. Compare the actual decisions. Earlier actions stay unchanged; new copy stays a draft.

Copy the complete prompt from `prompts.html#p3-manual-cycles`. Paste it into the same local Codex or Claude Code project.

If the required connection or initialized runtime is unavailable, save that exact gap in `company-motion/MANUAL-CYCLES.md` and name the next setup task. Do not claim the read or cycle completed; keep the finished local page play intact. Show the gap and next setup task, then stop here. Otherwise continue below.

Run two manual preparation/refresh/review cycles from this same project with new sending paused. Each cycle reads saved history, original pending IDs, actual exclusions, scoped mailbox history and bookings before selecting a next action.

Use the supported read commands and save actual timestamps, output and decisions in `company-motion/MANUAL-CYCLES.md`.

The second cycle must keep earlier sent/unknown actions unchanged. Do not resubmit or reimport them. New copy remains a draft without a new exact grant.

Show both cycles and verify the saved attempt counts and original keys.

Append this step, actual artifact, check and next task to `company-motion/PROGRESS.md`. Show the output. Stop. Next: pause and reopen the same project.

You should have:

- `MANUAL-CYCLES.md` with two actual timestamps and observed decisions.

Check before continuing:

- Two distinct manual cycles are observed.
- Original action keys and attempt counts remain intact.
- New copy stays local until separately approved.

If blocked: Use the missing operation as a concrete implementation task for your team or Metadata.

Next: pause and reopen the same project.

Pause. Reopen. Keep the history.

Reference: p3-pause. Main workshop.

Pause prevents new local commitments. Reopen the same project and check the original records. Existing provider queues need their own separate stop controls.

Time: 8 minutes. Difficulty: Easy.

Tools: Same local project and execution controls..

Do this:

1. Pause with the documented command. Read the actual local paused state.
2. Reopen the same project. Inspect the original records and history.
3. Run a read-only review. It must not make a new recipient-facing commitment.
4. Keep sending paused. Provider queues need separately approved native controls.

Copy the complete prompt from `prompts.html#p3-pause`. Paste it into the same local Codex or Claude Code project.

If the required connection or initialized runtime is unavailable, save that exact gap in `company-motion/RECOVERY.md` and name the next setup task. Do not claim the read or cycle completed; keep the finished local page play intact. Show the gap and next setup task, then stop here. Otherwise continue below.

Run pause and status through the saved project launcher. Confirm the actual local paused state. Reopen the same project and inspect the saved database, original action keys, attempts, exclusions and pending provider IDs.

Run a read-only preparation/review cycle while paused; do not make a new recipient-facing commitment.

Save `company-motion/RECOVERY.md` with the pause result and retained-history evidence.

If a provider queue or gift is still active, read its original object and show the specific provider stop/cancel control for my approval. Local pause alone does not cancel it.

Keep new sending paused at the end of this exercise.

Append this step, actual artifact, check and next task to `company-motion/PROGRESS.md`. Show the output. Stop. Next: check whether unattended operation is ready.

You should have:

- Pause and recovery evidence with retained history.

Check before continuing:

- Paused local execution cannot make a new commitment.
- Provider-side pending work is checked separately.
- Restart retains attempts and exclusions.
- Unknown outcomes cause readback rather than another send.

If blocked: If a real provider result remains unknown, keep that action held and name its provider read route.

Next: check whether unattended operation is ready.

Chapter 4: Prepare a schedule when ready

Readiness record. Schedule remains paused unless every gate passes.

Prepare a schedule. Activate only when ready.

Reference: `p3-schedule`. Main workshop.

Scheduling is optional. First verify complete current sources and exact ongoing authority. If those are missing, keep the proven play manual and save the missing setup.

Time: 10 minutes. Difficulty: Moderate.

Tools: Supported local automation feature; complete native sources only for unattended operation..

Do this:

1. Read both manual cycles. They must preserve earlier actions and records.
2. Check unattended prerequisites. Complete native customer, CRM, mailbox history and booking reads.
3. Prepare a paused schedule. Use an actually available local app tool.
4. Review before activation. New copy requires new exact authority.

Copy the complete prompt from [prompts.html#p3-schedule](#). Paste it into the same local Codex or Claude Code project.

Read both actual manual cycles and the unattended-operation guide.

Save company-motion/SCHEDULE-READINESS.md. Check complete fresh native customer, CRM, scoped/full configured mailbox history and booking reads, plus exact ongoing owner authority. Snapshot mode cannot support unattended operation.

If any gate is missing, name its exact setup task and keep our working play manual.

If ready and a real local scheduling tool is available, prepare one paused schedule for this same project.

Show actual workspace, timezone, wake instructions and first two scheduled times.

Each wake must read history and sources, prepare exact work, and release only separately approved keys. New copy is a draft until approved. Keep the Mac awake and online.

Do not activate a schedule from this prompt. Show it for my exact approval.

Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop. Next: record actual scheduled observations or leave them pending.

You should have:

- SCHEDULE-READINESS.md and a paused schedule only when the app supports it.

Check before continuing:

- No incomplete source is silently replaced by an old snapshot.
- Actual scheduling capability is checked.
- No schedule becomes active without exact recurring approval.

If blocked: Keep manual operation if native sources, scheduling support or exact recurring authority is missing.

Next: record actual scheduled observations or leave them pending.

Count observed runs, not planned runs

Reference: p3-two-cycles. Main workshop.

Two manual cycles are today's check. Scheduled wakes count after they happen. Save the exact next observation time instead of claiming an unobserved run completed.

Time: 10 minutes. Difficulty: Moderate.

Tools: Actual manual cycle records; optional active schedule only after separate approval..

Do this:

1. Open both manual cycle records. Check their actual timestamps and decisions.
2. Inspect any supported schedule. Leave it paused unless separately approved.
3. Save actual observations. Unobserved scheduled wakes remain pending.
4. Keep the original action history. No earlier action is resubmitted.

Copy the complete prompt from `prompts.html#p3-two-cycles`. Paste it into the same local Codex or Claude Code project.

Read `company-motion/MANUAL-CYCLES.md` and any actual supported schedule record.

Save `company-motion/CYCLE-OBSERVATIONS.md` distinguishing observed manual cycles from scheduled wakes.

If a schedule is still paused or missing exact recurring approval, leave it paused. Record that stage honestly.

If separately approved and active, observe only wakes that actually occur. Save their exact times, original action IDs and decisions; check no earlier action was resubmitted.

For a wake outside class, save its real next observation time in `company-motion/WAITS.md`. Do not manufacture a completed observation.

Keep pending provider outcomes tied to their original IDs.

Append this step, actual artifact, check and next task to `company-motion/PROGRESS.md`. Show the output. Stop. Next: record what you have actually built and what is still missing.

You should have:

- `CYCLE-OBSERVATIONS.md` with manual and scheduled outcomes labeled separately.

Check before continuing:

- Two manual cycles have real timestamps.
- Unobserved scheduled wakes stay pending.
- Earlier actions are not resubmitted.

If blocked: If no capacity remains, observe a read-only paused wake. It proves scheduling/history reads, not a fresh outreach cycle.

Next: record what you have actually built and what is still missing.

The framework supports more than today's one play

Reference: `p3-supported`. Main workshop.

Email, LinkedIn and gifts need different owned resources. Publication and ads are portable additions. Automatic replies and attendance still need adapters.

Chapter 5: Record results and choose the next play

Honest completion record. Extra channels are optional guides.

Your framework, working play and remaining gaps

Reference: `p3-checklist`. Main workshop.

Record the parts you built and their evidence. A working page or email play is the first result. Extra channels stay incomplete until their own checks pass.

Time: 8 minutes. Difficulty: Easy.

Tools: Codex and your workshop project..

Do this:

1. Open actual files and provider records. Use their IDs, dates and precise stages.
2. Save COMPLETION.md. Mark each capability from observed evidence.
3. Update WAITS.md. Keep original IDs, next reads and missing setup.

Copy the complete prompt from prompts.html#p3-checklist. Paste it into the same local Codex or Claude Code project.

Build company-motion/COMPLETION.md from actual files and checks.

First record core local completion: personal framework, company/target briefs, two reviewed pages, reused skill, paused authority/instructions, execution flow and prepared ten-company universe. Link each actual artifact and review result.

Separately record conditional email results: authenticated sender, buyer/validation evidence, exact self-test and prospect pilot stages. Record replies, bookings and provider cycles only when actually connected and observed. Missing access is a named gap; it does not fail a completed local page play. Record social, gifts, ads, public hosting and unattended operation as optional extensions. Keep automatic reply sending and attendance marked not wired. Never count queued, fixture or unobserved results as live.

Give every gap one next action and keep original pending IDs in company-motion/WAITS.md.

Append the actual check and next task to company-motion/PROGRESS.md. Show the output. Stop.

Next: finish the first remaining issue with individual help.

You should have:

- Evidence-linked full capability record.

Check before continuing:

- Core local completion is assessed separately from conditional email and operation.
- Each result has actual evidence and a date.
- Missing access stays a gap; scheduling stays optional.

If blocked: Use the missing operation as a concrete implementation task for your team or Metadata.

Next: finish the first remaining issue with individual help.

Finish one blocked operation at a time

Reference: p3-help. Main workshop.

Work individually. Open your capability record. Repair the first connection or failed check. Keep completed work and history.

Time: 30 minutes. Difficulty: Help.

Tools: Codex and your workshop project..

Do this:

Copy the complete prompt from [prompts.html#p3-help](#). Paste it into the same local Codex or Claude Code project.

Read company-motion/COMPLETION.md and the first incomplete main-lane check.
Repair an available local cause, rerun the required check and record the actual result.
Preserve working outputs, original keys, commitments and limits.
If only an optional extension remains, name its specific account, resource, cost and approval setup. Do not claim that instructions alone reproduce a live channel.
Append this step, actual artifact, check and next task to company-motion/PROGRESS.md. Show the output. Stop. Next: continue with the same saved project after class.

You should have:

- Repaired operation or exact remaining implementation task.

Check before continuing:

- The repaired check has observed evidence.

If blocked: Keep a named missing permission, funded account or implementation step. Share only the redacted progress record.

Next: continue with the same saved project after class.

Keep your working project. Add one channel at a time.

Reference: p3-take-home. Main workshop.

You keep the prepared framework, your company context and a verified play. Add the next channel when its account, resources and exact approval are ready.

Publish the page to your owned site

Reference: p3-publish. Optional extension.

Choose your owned hosting destination. Publish the exact reviewed file after approval. Open the actual URL and test its next step.

Optional after class. Difficulty: Moderate.

Tools: Owned Netlify token, fresh workshop site and reviewed standalone HTML. Browser access for desktop/phone checks..

Do this:

1. Show the reviewed HTML and destination. Use your own dedicated site.
2. Approve the exact publication. Check company, file and account.
3. Publish through the documented operation. Save the native deployment result.
4. Open desktop and phone views. Check the actual form or next-step link.

Copy the complete prompt from [prompts.html#p3-publish](#). Paste it into the same local Codex or Claude Code project.

Read API-CONNECTIONS.md, SETUP-SEQUENCE.md and the page-publishing guide.
 Use our reviewed standalone HTML and my fresh owned Netlify workshop site.
 The publisher uploads only index.html to that selected site.
 Run shasum -a 256 followed by the actual reviewed HTML path and read its file SHA-256.
 Run python3 -B gtm.py resource --operation publish-review --file followed by that same actual HTML path.
 Read the returned authenticated owner_id, site_id, file sha256 and approval_sha256.
 The approval_sha256 binds the site, owner and exact file bytes. It is different from the file SHA-256 alone.
 Show that exact review and intended destination.
 After I approve that publication, run python3 -B gtm.py resume.
 Run python3 -B gtm.py resource --operation publish --file followed by the same actual path and --approved-sha256 followed by the returned approval_sha256.
 Run python3 -B gtm.py pause.
 Save the provider deploy ID and actual URL.
 Open that URL at 1440 × 900 and 390 × 844.
 Match it to the reviewed content and test its next-step destination.
 For a form, submit only my authorized controlled details and verify its actual destination record.
 Save company-motion/PUBLISH-RESULT.md.
 If deployment is pending, keep the original ID and next read in company-motion/WAITS.md.
 Repeat the same known deployment read instead of uploading a replacement.
 A successful upload does not prove a working form.
 Use the saved verified interpreter for standalone scripts. Show the actual result and keep any missing stage visible. Stop. Next: record this optional extension in company-motion/COMPLETION.md.

You should have:

- Native deployment result and verified public page.

Check before continuing:

- Returned approval binds owned site, owner and exact bytes.
- Changed site or file is held before upload.
- Actual HTTPS URL shows the reviewed bytes and readable layouts.
- A form has separate submission evidence when used.

If blocked: Keep the local reviewed page if hosting is unavailable. Mark publication incomplete.

Next: record this extension's actual stage in COMPLETION.md.

LinkedIn: from tabs to a checked queue

Reference: p3-social-before-after. Optional extension.

Before: find profiles, connect, write messages and track replies in tabs. After: Codex prepares the contact; Gojiberry runs the approved stage; v5 reads the result.

Next: record this extension's actual stage in COMPLETION.md.

Verify your LinkedIn seat, list and campaign

Reference: p3-social-setup. Optional extension.

Create an isolated list and invitation-only campaign in Gojiberry. Prepare one paused test contact. Check the exact owned IDs. Keep every other contact paused.

Optional after class. Difficulty: Moderate.

Tools: Gojiberry paid account and owned LinkedIn seat/list/campaign..

Do this:

1. Open your owned provider dashboard. Create an isolated list and inactive invitation-only campaign.
2. Read the exact IDs. Match the seat, campaign and list to your account.
3. Prepare one paused contact. This creates a real provider object after your approval.
4. Inspect every campaign list. All contacts must stay paused before activation.

Copy the complete prompt from prompts.html#p3-social-setup. Paste it into the same local Codex or Claude Code project.

Read API-CONNECTIONS.md, SETUP-SEQUENCE.md and my private Gojiberry config.
Open my owned Gojiberry dashboard through the authorized browser.
Create a dedicated workshop list and inactive invitation-only campaign in the provider UI.
Use my actual owned LinkedIn seat.
Automatic list creation and whole-campaign activation are disabled in this runtime.
Read the exact seat, campaign, list, campaign name and steps through supported provider reads.
Save these actual identities in settings.social_owned under the documented sender label.
Configure the actual private file while preserving credentials and prior limits; execution stays paused.
Show the exact contact change and obtain my approval before creating its provider object.
Write the documented create_contact input privately with the exact owned listId.
Run `python3 -B gtm.py social-provision --operation create_contact --input` followed by that actual private path.
Read the actual returned contact ID and confirm state:"paused", readyForCampaign:false, exact profile, email and list.
Verify every contact in every list of the dedicated campaign remains paused.
Keep the campaign inactive during contact preparation.
Save company-motion/SOCIAL-REVIEW.md with the owned IDs, exact steps, contact and intended release.
Keep uncertain provisioning handles and next reads in company-motion/WAITS.md.
Do not repeat an uncertain create_contact request.
For uncertain provisioning, read the owned Gojiberry contact/list UI using the original returned ID when known.
Match the exact profile/email, list and request time from our saved request.
Use a supported get_contact read only when the contact ID is known.
This provisioning request is not an outreach action for gtm.py reconcile.
If its original object cannot be positively identified, retain the unknown state and do not create it again.
Record the actual UI/read route in company-motion/WAITS.md.
Use the saved verified interpreter for standalone scripts. Show the actual result and keep any missing stage visible. Stop. Next: record this optional extension in company-motion/COMPLETION.md.

You should have:

- One verified paused contact and exact social review.

Check before continuing:

- Dedicated campaign/list/seat belong to your account.
- Campaign has invitation steps only and remains inactive during preparation.

- Exact created contact is paused with readyForCampaign:false.
- All campaign lists contain only paused contacts.

If blocked: Without owned resources, save the exact social plan. Mark social as not connected.

Next: record this extension's actual stage in COMPLETION.md.

Release one approved LinkedIn stage

Reference: p3-social-run. Optional extension.

Approve the isolated campaign in Gojiberry. Release one exact paused contact through the runtime. Read its provider stage. A queue is still queued.

Optional after class. Difficulty: Moderate.

Tools: Owned Gojiberry seat, isolated provider UI campaign, one paused test contact and exact one-stage approve-scope grant..

Do this:

1. Approve the exact stage. Use your willing contact and owned seat.
2. Prepare one guarded action. Use the supported schema and command.
3. Run the exact approved key once. Use the action-key selector and a limit of one.
4. Read the native result. A message waits for real invitation acceptance.

Copy the complete prompt from prompts.html#p3-social-run. Paste it into the same local Codex or Claude Code project.

Read company-motion/SOCIAL-REVIEW.md and the social action schema.

Use the actual contact_id and saved successful adapter creation receipt from the previous social-provision create_contact operation.

An arbitrary contact ID or manually created contact cannot replace that original creation receipt.

After importing its exact action, verify the saved social_prepared record matches that contact ID, original action key and payload.

A manually created paused contact is not the workshop's prepared contact and cannot replace this record.

Show the exact owned seat, invitation-only campaign/list and willing test contact.

After I approve that exact scope, set linkedin_dispatch:"automatic", social daily target:1 and social batch limit:1 in my actual private config.

Preserve my Google credential paths, sender limits and exclusions.

Run configure with that actual config; execution stays paused.

Write company-motion/SOCIAL-ACTION.json with only the exact reviewed action.

Import it with `python3 -B gtm.py import --kind actions --file ../company-motion/SOCIAL-ACTION.json`.

Read the actual returned action key.

After I approve that key and exact stage, write company-motion/SOCIAL-APPROVAL.json.

Use authority:my actual approval, purpose:"one invitation-only workshop test", action_keys:[that key], limits:{email:0,gift:0,social:1}.

Run `python3 -B gtm.py approve-scope --input ../company-motion/SOCIAL-APPROVAL.json`.

Read every list/contact in this dedicated owned campaign.

After I separately approve that exact isolated campaign activation, activate it through the provider UI while every contact remains paused.

Read its actual active state; local runtime will not activate the campaign.

Refresh current sources and review the exact action's checks.

After I approve the exact contact release, run `python3 -B gtm.py resume`.

Run `python3 -B gtm.py run --limit 1 --action-key ORIGINAL_KEY` once.

Replace ORIGINAL_KEY with this exact imported social action key from its approved scope receipt.

The selector must not release an email, gift or different social action from another ready grant.

Run `python3 -B gtm.py pause`.

Reconcile the original key and save company-motion/SOCIAL-RESULT.md.

Keep a verified provider queue labeled queued until its real sent event exists.

Keep all other contacts paused.

A later message needs actual acceptance and a new exact supported stage approval.

Save pending stages and next reads in company-motion/WAITS.md.

Local pause does not cancel a provider queue; inspect its actual campaign/contact stop controls.

Use the saved verified interpreter for standalone scripts. Show the actual result and keep any missing stage visible. Stop. Next: record this optional extension in company-motion/COMPLETION.md.

You should have:

- One provider social result and action key.

Check before continuing:

- Exact grant allows one supported social stage only.
- Provider contact and campaign match that approval.
- Queued is not counted as sent.
- Recipient waits keep the original action key.
- The manual command selects the exact original approved social key, without releasing another grant.

If blocked: If provider restrictions or recipient acceptance block the stage, keep that state and next read time. Other channels can continue.

Next: record this extension's actual stage in COMPLETION.md.

Gifting: one offer, exact terms, provider proof

Reference: p3-gift-before-after. Optional extension.

Before: select a recipient, build the gift, copy terms and check delivery manually. After: prepare the exact offer once, send through checks and read its provider stage.

Next: record this extension's actual stage in COMPLETION.md.

Check the collection, funding and meeting gate

Reference: p3-gift-setup. Optional extension.

The Loop & Tie key is only the connection. Verify your team, collection, balance and exact meeting requirement before preparing the gift.

Optional after class. Difficulty: Moderate.

Tools: Loop & Tie account, funded owned team, collection and verified native meeting gate..

Do this:

1. Read your gift team and balance. Check the real collection cost.
2. Verify the meeting gate. Read its native numeric ID and binding.
3. Write truthful terms. Include value, expiry and meeting requirement.
4. Preview the exact recipient. Use one approved gift recipient.

Copy the complete prompt from [prompts.html#p3-gift-setup](#). Paste it into the same local Codex or Claude Code project.

Read API-CONNECTIONS.md, SETUP-SEQUENCE.md and GIFT-GATE-HELPER.md.
 Use my owned Loop & Tie team, actual funded collection and meeting scheduler.
 Open the exact scheduler Edit page in my signed-in browser.
 Save the actual full HTML privately outside this repository, with mode 0600.
 Record its actual HTTPS capture_url and captured_at. Never manufacture or refresh a timestamp.
 For the current Edit Calendar page, use ui_contract:"scheduler-edit-v1".
 Match the exact Edit URL and POST action, numeric edit_scheduler_ID form, scheduler[name] and scheduler[url] to the authenticated native catalogue.
 Read the visible statement that a meeting is required to redeem the gift.
 The older input-control contract applies only if those actual inputs exist. Do not invent them.
 Fill api-templates/gift-gate.template.json from the real capture. Keep the owned team and provider identity.
 Import from code/: `python3 -B gtm.py import --kind gift-gate --file ACTUAL_PRIVATE_JSON_PATH`
 Replace ACTUAL_PRIVATE_JSON_PATH with that saved file. Capture must be within 65 minutes.
 Show the actual import result. If stale, recapture the real page.
 Read the actual collection price, resolved provider currency, funding and shipping permission.
 Prepare one gift preview in company-motion/GIFT-REVIEW.md.
 Show the exact recipient, collection, value, schedule, expiry, subject, body and meeting requirement.
 Keep credentials, funding details and recipient records private.
 Do not create a gift until I approve its exact recipient, terms and cost.
 Use the saved verified interpreter. Keep every missing stage visible.
 Show the output. Stop. Next: review the gift and approve its exact scope.

You should have:

- Verified gift prerequisites and exact gift preview.

Check before continuing:

- Actual team, catalogue and form identify the same owned scheduler.
- Actual scheduler Edit URL, numeric form and visible meeting requirement match its native catalogue.
- Signed-in capture URL/team match and actual capture is within 65 minutes.
- Collection cost/currency, value and expiry are truthful.

If blocked: Without a funded team or provider gate proof, keep the gift prepared and label the missing prerequisite.

Next: record this extension's actual stage in COMPLETION.md.

Create one approved gift. Read its stage.

Reference: p3-gift-run. Optional extension.

Approve the recipient, offer and cost. Run the exact action once. Read the provider gift ID and events. Redemption may happen later.

Optional after class. Difficulty: Moderate.

Tools: Funded Loop & Tie team and approved one-gift scope. Real collection charge may apply..

Do this:

1. Approve the exact gift and cost. Use the saved preview.

2. Run the exact approved gift key once. Use the action-key selector and a limit of one.
3. Read the native gift. Match recipient, collection, copy and expiry.
4. Save the provider stage. Created, sent and redeemed differ.

Copy the complete prompt from prompts.html#p3-gift-run. Paste it into the same local Codex or Claude Code project.

Show company-motion/GIFT-REVIEW.md and its exact recipient payload.
 Read the owned collection's actual price. Use its explicit currency, or the documented Loop & Tie USD credit contract through the supplied adapter. Keep its pricing source in the review.
 After I approve this offer and cost, set gift_fulfillment:"automatic", gift daily target:1 and gift batch limit:1 in my actual private config.
 Keep gift_use_full_balance:false; this workshop does not grant the provider's entire balance.
 Preserve all credential paths, prior limits and exclusions.
 Configure that actual private file; execution remains paused.
 Write company-motion/GIFT-ACTION.json with only this exact gift.
 Import with `python3 -B gtm.py import --kind actions --file ../company-motion/GIFT-ACTION.json`.
 Read its actual returned action key.
 After I approve that key and cost, write company-motion/GIFT-APPROVAL.json.
 Use authority:my actual approval, purpose:"one workshop gift", action_keys:[that key], limits:{email:0,gift:1,social:0}.
 Set gift_budget.currency to the actual collection currency.
 Set gift_budget.max_per_gift and gift_budget.max_total to my approved actual cost ceiling.
 Use numeric amounts and the actual ISO currency, not a default invented price.
 Run `python3 -B gtm.py approve-scope --input ../company-motion/GIFT-APPROVAL.json`.
 Hold the gift when currency cannot be resolved by the provider contract or differs from the grant.
 Refresh current sources and review all gift checks.
 The actual meeting-gate form capture must still be within 65 minutes.
 If stale, re-capture the owned form and reimport only that fresh gift-gate evidence, preserving the original gift action key.
 Never alter the old capture timestamp to make it current.
 Run `python3 -B gtm.py resume`.
 Run `python3 -B gtm.py run --limit 1 --action-key ORIGINAL_KEY` once.
 Replace ORIGINAL_KEY with this exact imported gift action key from its approved scope receipt.
 The selector must not release an email, social action or different gift from another ready grant.
 Run `python3 -B gtm.py pause`.
 Reconcile that original key.
 Save company-motion/GIFT-RESULT.md with the provider gift ID, exact matched fields, actual price/currency, committed budget and stage.
 Keep created_verified, sent and redeemed separate.
 If creation is uncertain, read the existing commitment rather than creating another gift.
 Save pending stages and next reads in company-motion/WAITS.md.
 Use the saved verified interpreter for standalone scripts. Show the actual result and keep any missing stage visible. Stop. Next: record this optional extension in company-motion/COMPLETION.md.

You should have:

- One provider gift ID and exact provider stage.

Check before continuing:

- Immutable grant names this exact recipient, terms and key.
- Actual collection price/currency fit the per-gift and lifetime ceilings.

- Only one gift is authorized.
- Prior gifts and unknown commitments are not repeated.
- The manual command selects the exact original approved gift key, without releasing another grant.

If blocked: Provider or recipient waits continue later. Keep the original ID and next read action.

Next: record this extension's actual stage in COMPLETION.md.

Recorded MQLs: one to two. Pipeline unverified.

Reference: p3-ad-results. Optional extension.

Recorded MQLs rose from one to two. Pipeline impact was unverified. Audit the 43 leads and meeting outcomes before increasing total spend.

Next: record this extension's actual stage in COMPLETION.md.

Turn your ad results into one clear action

Reference: p3-ad-review. Optional extension.

Read your real ad account. Compare equal periods. Save three recommendations and one creative brief. Keep campaign changes for the next exact approval.

Optional after class. Difficulty: Easy.

Tools: Own Metadata/advertising connection or official ad export. Creative credits only if approved generation is used..

Do this:

1. Read your own ad account. Use a connected Metadata account or an official export.
2. Compare complete equal periods. Keep spend, clicks, leads and qualification separate.
3. Save three concrete improvements. Use the actual experiment evidence.
4. Prepare one creative brief. Keep launch and budget changes for separate approval.

Copy the complete prompt from prompts.html#p3-ad-review. Paste it into the same local Codex or Claude Code project.

Read my connected advertising account or the official export I provide.
 Compare the last 30 complete days with the prior 30.
 Save company-motion/AD-REVIEW.md with spend, impressions, clicks, leads and available qualification, including dates, currency and denominator.
 Recommend three specific changes supported by actual campaign evidence.
 Prepare one creative brief using our sourced company brand and offer.
 If my Metadata creative connection is configured, show its supported exact request and actual owned account.
 After I approve that exact creative request and cost scope, resume.
 Run `python3 -B gtm.py resource --operation generate --input` followed by the actual private input path once.
 Pause afterward and save the actual returned creative for visual review.
 A pending generation keeps its original request key.
 The original v5 source has no campaign activation adapter.
 This portable kit adds explicit owned-account campaign operations.
 They require separate ad authority and exact approval.
 They are not automatically added to the hourly outreach schedule.
 Do not launch or change spend.
 For uncertain creative generation, preserve the original local request key and any provider/request ID.
 Open the owned Metadata creative/request view and inspect the original operation, domain and time when that view is available.
 If the provider does not expose that read, record the exact missing read route and retain the unknown request.
 Generic gtm.py reconcile does not reconcile creative requests.
 Do not run generation again to find out what happened.
 Use the saved verified interpreter for standalone scripts. Show the actual result and keep any missing stage visible. Stop. Next: record this optional extension in company-motion/COMPLETION.md.

You should have:

- Real ad comparison, three recommendations and one reviewed creative brief.

Check before continuing:

- Periods and denominators match.
- No pipeline attribution is invented.
- Generated creative is not called a live campaign.

If blocked: Use your official ad export. Without account data, save the specific missing read operation and do not fabricate performance.

Next: record this extension's actual stage in COMPLETION.md.

Build one ad draft. Review it in Metadata.

Reference: p3-ad-test. Optional extension.

Use your owned audience, offer, creative and budget. Save a reviewed draft. Launch and restart need a separate review in the Metadata dashboard.

Optional after class. Difficulty: Moderate.

Tools: Own Metadata account/token, reviewed dedicated campaign, native readiness and separate approved advertising budget. Real spend may apply..

Do this:

1. Read your real account and budget group. Use owned resources and actual spending ceilings.
2. Review the exact draft request. Audience, offer and creative must match your approval.
3. Create only the approved draft. Read its provider ID and non-running state.
4. Inspect it in Metadata. A later launch requires separate exact approval.

Copy the complete prompt from `prompts.html#p3-ad-test`. Paste it into the same local Codex or Claude Code project.

Read the portable advertising guide and current captured provider schemas.
Use my owned Metadata account, connected ad platform and dedicated workshop draft.
Read the actual integrations, audience, offer, creative and owned budget group.
Save the provider budget-group values and my approved daily and total ceilings.
Populate policy.advertising with my actual authority and budget ceilings.
Preserve existing sender authority, credential paths and exclusions.
Configure the actual private file; execution stays paused.
Prepare company-motion/AD-DRAFT.json for the supported create_campaign operation using its current exact schema.
Use a dedicated name and owned budgetGroup.
Only create a draft when the inspected provider contract confirms this operation cannot start spend.
Otherwise save that draft in the owned Metadata UI and record the contract gap.
Run `python3 -B gtm.py ads-review --operation create_campaign --input ../company-motion/AD-DRAFT.json`.
Show the actual account, audience, offer, creative, budgets and returned approval hash.
After I approve this exact non-running draft creation, resume and run `python3 -B gtm.py ads --operation create_campaign --input ../company-motion/AD-DRAFT.json --approved-sha256` followed by that returned hash.
Pause after the operation.
Read the actual campaign by its name and record its provider ID and state.
CLI launch and restart are disabled until existing-budget contracts are verified.
Open the exact owned draft in Metadata and inspect its audience, copy, offer, integration readiness and exact spending limits.
Do not start spend from this workshop draft prompt.
A later UI launch needs my separate exact campaign and budget approval plus positive current readiness.
Save company-motion/AD-TEST.md and any original pending request in company-motion/WAITS.md.
Do not call a created draft or queued response running.
For an uncertain draft request, preserve its original local key and any provider ID.
Use ads-review with search_campaigns_by_names and the exact previously approved unique campaign name.
Read the owned Metadata draft UI and match its account, name, creation time and approved fields.
Generic gtm.py reconcile does not reconcile advertising requests.
A missing search result is not proof of failed creation. Keep unknown outcomes held without another create request.
Save the precise provider/UI read route in company-motion/WAITS.md.
Use the saved verified interpreter for standalone scripts. Show the actual result and keep any missing stage visible. Stop. Next: record this optional extension in company-motion/COMPLETION.md.

You should have:

- Exact approved draft receipt, owned Metadata UI review and actual campaign state.

Check before continuing:

- Owned account, audience and budget group match the review.
- Draft creation cannot start spend under the inspected provider contract.
- Daily and total budgets fit the separate owner authority.
- Launch/restart CLI remains held.
- Draft is not labeled a running campaign.

If blocked: Without a ready owned account or verified draft contract, keep the exact draft and missing setup. Do not start spend.

Next: record this extension's actual stage in COMPLETION.md.

Finish with an honest completion record

Core local work:

- Prepared personal framework with a verified compatible interpreter
- Sourced company brief and two real target briefs
- Two reviewed local account pages and one successfully reused skill
- Readable company instructions and paused authority
- Saved execution flow with source checks and next-action rules
- Prepared ten-company target universe with source, fit and country evidence
- Dated completion record with actual stages and one next action per gap

When your email connections pass

- Own authenticated sender/calendar, current exclusions and exact country scope
- Current employer and valid exact email evidence for every released address
- Original exact approval for one owned self-test and ten prospect messages
- One self-test matched exactly in Gmail Sent before any prospect release
- Each prospect result labeled accurately; ten exact prospect Sent matches for a full pilot, fewer labeled partial

When your operating connections pass

- Actual reply and booking reads when connected, including honest empty reads
- Two observed manual source/review cycles when source access is available
- Retained history and a verified local pause when the runtime is initialized

Optional extensions

- Verified public page and next-step destination on an owned site

- Owned LinkedIn resources and one exact provider stage
- Funded gift with truthful terms and one exact provider stage
- Equal-period advertising review and a separately approved non-running draft
- Complete native exclusion/history/booking coverage and actual observed scheduled wakes

Limits to keep visible

- One working play does not establish full live replication of every channel.
- The prepared framework saves setup work. It does not supply company accounts, keys, consent or spending authority.
- An exact Gmail Sent match proves sending, not inbox delivery, pipeline or a meeting.
- Automatic email reply sending and meeting attendance attribution are not wired.
- New copy, recipients or channels need their own bounded owner approval.
- Social queued, gift created, gift sent and gift redeemed are distinct provider stages.
- Owned-site publishing and ad draft operations are portable workshop additions.
- CLI advertising launch/restart remains held. An owned UI launch needs separate exact account and budget approval.
- Unattended operation requires complete fresh native customer, CRM, mailbox-history and booking reads. An old snapshot cannot sustain it.
- Local pause prevents new local commitments. It does not cancel provider queues or existing gifts, deployments or delayed ad operations.
- Scheduled wakes count only after they actually occur.

Fix the first blocked step

Copy this when a step fails. Keep tokens and private recipient records out of the error.

Read my project instructions and company-motion/PROGRESS.md. Identify the current workshop step and first failed check. Read the actual error and existing files. Fix that one problem without replacing completed work, changing authority or resending an uncertain provider action. Rerun the check. Show the actual result and next exercise, then stop.

Error: [PASTE THE ERROR WITHOUT KEYS OR PRIVATE RECIPIENTS]

Words you may need

Word	Meaning here
Coding agent	Codex or Claude Code, working with files on your computer.
Company brief	A saved description of your product, buyers, offer and source-backed proof.
Skill	A saved set of instructions your agent can reuse, such as building an account page.
Execution flow	The order of work and the conditions for continuing or stopping.
Runtime	The local program that checks prepared actions and records results.
API	A connection that lets a program use a tool, such as Loop & Tie.
API key	A private password for that connection.
OAuth	A browser sign-in that grants specific account access.
Authority	The exact actions, recipients and spending you have approved.
Exclusion	A company or person the workflow must not contact.
Action key	The saved identifier for one exact prepared action.
Self-test	One exact email to your own authorized sender.
Receipt	A saved response or native record showing what actually happened.
Held	Stopped because a required fact, connection or approval is missing.
Reconcile	Read the original provider result without sending another copy.
Queued	Waiting with the provider. It has not yet been confirmed sent.
Booked	A meeting reservation. It does not show attendance.

What the evidence supports

The real redacted examples are Metadata account pages, email and gift invitation records. The September 30 company dashboard: 93 demo requests, 64 booked and 34 completed does not attribute all outcomes to this engine. The historical ten-account pilot matched ten exact Gmail Sent records and blocked ten repeat attempts. See [sources/V5-WORKSHOP-SOURCES.md](#) for dates and denominators.

A prior participant reported completing Part 1 in Claude Code. Software checks and model reviews on this revision are separate from an independent beginner trial. Do not claim live provider success

from these checks.

Automatic reply sending, meeting scheduling writes, physical post, whole social campaign activation and ad launch/restart are not implemented here. Scheduling requires maintained sources, two observed manual cycles, actual app support and explicit recurring authority. Otherwise leave it prepared and paused.